

1)Implementation of DECISION TREE

ALGORITHM:

CODE:

```
import pandas as pd
```

```
import math
```

```
# -----
```

```
# Dataset: Salary, Age, Experience
```

```
# -----
```

```
data = {
```

```
    'Salary':
```

```
['Low','Low','Medium','High','High','Medium','Low',  
'High'],
```

```
    'Age':
```

```
['Young','Young','Middle','Old','Middle','Old','Middle',  
'Young'],
```

```
    'Experience':
```

```
['No','Yes','Yes','Yes','No','Yes','No','No'],
```

```
    'BuyLaptop':
```

```
['No','Yes','Yes','Yes','Yes','Yes','No','Yes']
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# -----
```

```
# Entropy Function
```

```
# -----
```

```
def entropy(target):
```

```
    values = target.value_counts()
```

```
    total = len(target)
```

```

ent = 0

for count in values:

    p = count / total

    ent -= p * math.log2(p)

return ent

```

```

# -----

```

```

# Information Gain

```

```

# -----

```

```

def information_gain(data, attribute, target):

```

```

    total_entropy = entropy(data[target])

```

```

    values = data[attribute].unique()

```

```

    weighted_entropy = 0

```

```

    for value in values:

```

```

        subset = data[data[attribute] == value]

```

```

        weighted_entropy += (len(subset) /
len(data)) * entropy(subset[target])

```

```

    return total_entropy - weighted_entropy

```

```

# -----

```

```

# ID3 Algorithm

```

```

# -----

```

```

def id3(data, original_data, features, target,
parent_node=None):

```

```

if len(data[target].unique()) == 1:
    return data[target].iloc[0]

elif len(data) == 0:
    return original_data[target].mode()[0]

elif len(features) == 0:
    return parent_node

else:
    parent_node = data[target].mode()[0]

    gains = [information_gain(data, feature,
target) for feature in features]

    best_feature =
features[gains.index(max(gains))]

    tree = {best_feature: {}}

    remaining_features = [f for f in features if f
!= best_feature]

    for value in data[best_feature].unique():
        subset = data[data[best_feature] ==
value]

        subtree = id3(
            subset,
            original_data,

```

```
        remaining_features,  
        target,  
        parent_node  
    )
```

```
    tree[best_feature][value] = subtree
```

```
    return tree
```

```
# -----
```

```
# Build Decision Tree
```

```
# -----
```

```
features = list(df.columns[:-1])
```

```
tree = id3(df, df, features, 'BuyLaptop')
```

```
print("Decision Tree:")
```

```
print(tree)
```

```
# -----
```

```
# Prediction Function
```

```
# -----
```

```
def predict(query, tree, default="Yes"):
```

```
    for key in query.keys():
```

```
        if key in tree:
```

```
            try:
```

```

        result = tree[key][query[key]]

    if isinstance(result, dict):
        return predict(query, result)
    else:
        return result
except:
    return default

# -----
# New Sample
# -----

new_sample = {
    'Salary': 'Medium',
    'Age': 'Young',
    'Experience': 'Yes'
}

prediction = predict(new_sample, tree)

print("\nNew Sample:")
print(new_sample)

print("\nPrediction:", prediction)

```

OUTPUT:

OUTPUT:

```
*** Decision Tree:
{'Salary': {'Low': {'Experience': {'No': 'No', 'Yes': 'Yes'}}, 'Medium': 'Yes', 'High': 'Yes'}}

New Sample:
{'Salary': 'Medium', 'Age': 'Young', 'Experience': 'Yes'}

Prediction: Yes
```